# Qdisc Notes

John Fastabend, Intel

# Qdisc Lock

- pfifo_fast
  Current: 1: 1469k
  4: 1120k
  8: 999k
  No_lock: 1: 1418k
  4: 1804k
  8: 1358k
  No_lock* (cmpxch)
  1: 1520k

- Next Steps:
  - ONETXQUEUE only
  - Implies -> check queue_stop before ever doing a dequeuer

- Future: dequeue multiple skbs in one step
  xmit enqueue multiple skbs

# Match Tool

# # ./matchd

```
# ./tc qdisc show dev enp1s0f1
qdisc ingress ffff: parent ffff:fff1 ----------------

# ./tc filter show  dev enp1s0f1 parent ffff:
filter protocol ip pref 99 u32
filter protocol ip pref 99 u32 fh 2: ht divisor 1
filter protocol ip pref 99 u32 fh 1: ht divisor 1
filter protocol ip pref 99 u32 fh 800: ht divisor 1
filter protocol ip pref 99 u32 fh 800::1 order 1 key ht 800
bkt 0 link 1:
  match 00060000/00ff0000 at 8
    offset 0f00>>6 at 0  eat
filter protocol ip pref 99 u32 fh 800::3 order 3 key ht 800
bkt 0 link 2:
  match 00110000/00ff0000 at 8
    offset 0f00>>6 at 0  eat
```

```
# ./match set_rule prio 1 handle 1 table 1 match tcp.dst_port 23
action drop
table : 1  uid : 1  prio : 1  bytes : 0  packets : 0
         tcp.dst_port = 0017 (ffff)
           1: drop (  )


# ./tc filter show  dev enp1s0f1 parent ffff:
filter protocol ip pref 99 u32
filter protocol ip pref 99 u32 fh 2: ht divisor 1
filter protocol ip pref 99 u32 fh 1: ht divisor 1
filter protocol ip pref 99 u32 fh 1::2 order 2 key ht 1 bkt 0
terminal flowid ???
  match 00000017/0000ffff at nexthdr+0
        action order 1: gact action drop
         random type none pass val 0
         index 1 ref 1 bind 1

filter protocol ip pref 99 u32 fh 800: ht divisor 1
filter protocol ip pref 99 u32 fh 800::1 order 1 key ht 800 bkt 0
link 1:
  match 00060000/00ff0000 at 8
    offset 0f00>>6 at 0  eat
filter protocol ip pref 99 u32 fh 800::3 order 3 key ht 800 bkt 0
link 2:
  match 00110000/00ff0000 at 8
    offset 0f00>>6 at 0  eat
```

# P4 Parse Graph Definition Language

```
header type Ethernet_t {
    fields {
        dstAddr : 48;
        srcAddr : 48;
        etherType : 16;
    }
}

header_type ipv4_t {
    fields {
            ….
    }
}
```

www.P4.org/spec

# P4 Parse Graph Definition Language

```
parser ethernet  {
        extract(ethernet);
        return select(latest.ethertype) {
                0x8100 : parse_vlan;
                0x800   : parse_ipv4;
                default  : ingress;
        }
}

parser  ipv4 {
        extract(ipv4)
        return select(latest.protocol) {
                IP_PROTOCOL_UDP : parse_udp;
                IP_PROTOCOL_TCP :  parse_tcp;
        }
}
```

www.P4.org/spec

# Autogenerated *models.h

```
static struct net_mat_jump_table u32_parse_ethernet[] =
{
        {   .node = HEADER_INSTANCE_IPV4,
            .field = {
                    .header = HEADER_ETHERNET,
                    .field = HEADER_ETHERNET_ETHERTYPE,
                    .type = NET_MAT_FIELD_REF_ATTR_TYPE_U16,
                    .v.u16 = {
                            value_u16 = 0x0800,
                            .mask_u16 = 0xFFFF,
                    }
        }
},
[...]
```

Would others find this useful?